

Java 16 — Records

Java Version: 16

Year: 2021

Feature: Records (Final)

UniqueClassName: Records

Definition

Records became a **final feature in Java 16**. They provide a compact way to create classes whose primary purpose is to **store immutable data**.

Instead of manually writing:

- private fields
- constructor
- getter/accessor methods
- equals()
- hashCode()
- toString()

a record generates these automatically.

Basic Syntax

```
public record Person(int id, String name, String email) {  
}
```

This automatically provides:

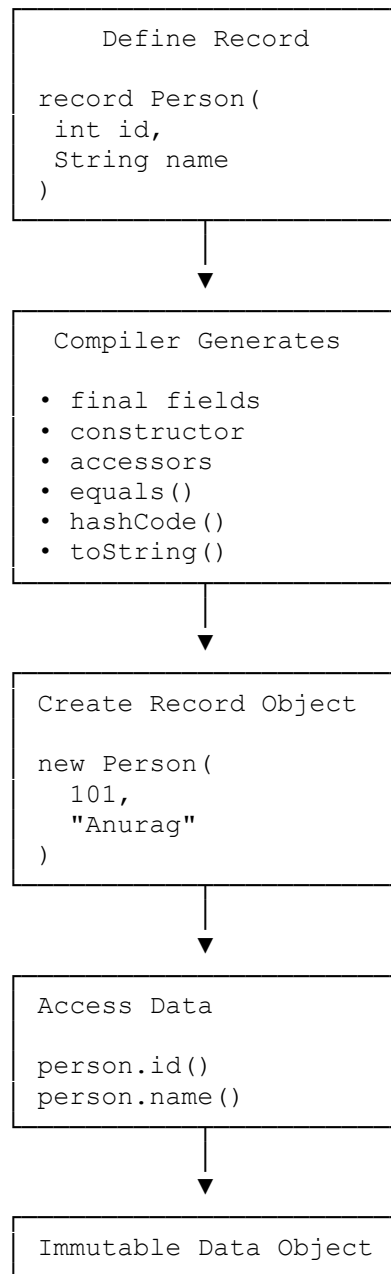
```
private final int id;  
private final String name;  
private final String email;
```

Constructor

```
id()  
name()  
email()  
equals()  
hashCode()  
toString()
```

Record accessors are named `id()`, `name()`, and `email()`, rather than traditional JavaBean-style `getId()`, `getName()`, etc.

Flowchart



Java Code Example

```
public class Records {  
  
    // Java 16 record  
    public record Person(  
        int id,  
        String name,  
        String email
```

```

    } {
    }

    public static void main(String[] args) {

        // Create record object
        Person person = new Person(
            101,
            "Anurag",
            "anurag@example.com"
        );

        // Access record components
        System.out.println("ID      : " + person.id());
        System.out.println("Name    : " + person.name());
        System.out.println("Email   : " + person.email());

        // Automatically generated toString()
        System.out.println("Person : " + person);

        // Automatically generated equals()
        Person person2 = new Person(
            101,
            "Anurag",
            "anurag@example.com"
        );

        System.out.println("Equal   : " + person.equals(person2));

        // Automatically generated hashCode()
        System.out.println("Hash     : " + person.hashCode());
    }
}

```

Output

```

ID      : 101
Name    : Anurag
Email   : anurag@example.com
Person  : Person[id=101, name=Anurag, email=anurag@example.com]
Equal   : true
Hash    : <generated hash value>

```

Record with Validation

Records can also contain a **compact constructor** for validation:

```

public record Employee(int id, String name, double salary) {

    public Employee {
        if (id <= 0) {
            throw new IllegalArgumentException("ID must be positive");
        }
    }
}

```

```

        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name is required");
        }

        if (salary < 0) {
            throw new IllegalArgumentException("Salary cannot be negative");
        }
    }
}

```

Usage:

```

public class Records {

    public record Employee(int id, String name, double salary) {

        public Employee {
            if (id <= 0) {
                throw new IllegalArgumentException("Invalid ID");
            }

            if (name == null || name.isBlank()) {
                throw new IllegalArgumentException("Name is required");
            }
        }
    }

    public static void main(String[] args) {

        Employee employee =
            new Employee(101, "Anurag", 75000);

        System.out.println(employee.id());
        System.out.println(employee.name());
        System.out.println(employee.salary());
    }
}

```

Key Characteristics

Feature	Record
Introduced	Java 14 Preview
Final	Java 16
Primary purpose	Data-carrying classes
Fields	private final
Constructor	Generated

Accessors	Generated
equals()	Generated
hashCode()	Generated
toString()	Generated
Mutable components	No reassignment
Can implement interfaces	Yes
Can extend another class	No
Can contain methods	Yes

In short: Java 16 Records reduce boilerplate when a class primarily represents **immutable data**.

Java 16 — Pattern Matching for instanceof

Java Version: 16

Year: 2021

Feature: Pattern Matching for instanceof — **Final**

UniqueClassName: InstanceofFinal

Definition

Pattern Matching for instanceof became a **final feature in Java 16**.

It simplifies type checking and casting by combining both operations into a single expression.

Before Java 16

You typically had to:

```
if (obj instanceof String) {
    String str = (String) obj;
    System.out.println(str.length());
}
```

There are two separate operations:

1. Check whether obj is a String.
2. Explicitly cast obj to String.

Java 16

Pattern matching eliminates the explicit cast:

```
if (obj instanceof String str) {  
    System.out.println(str.length());  
}
```

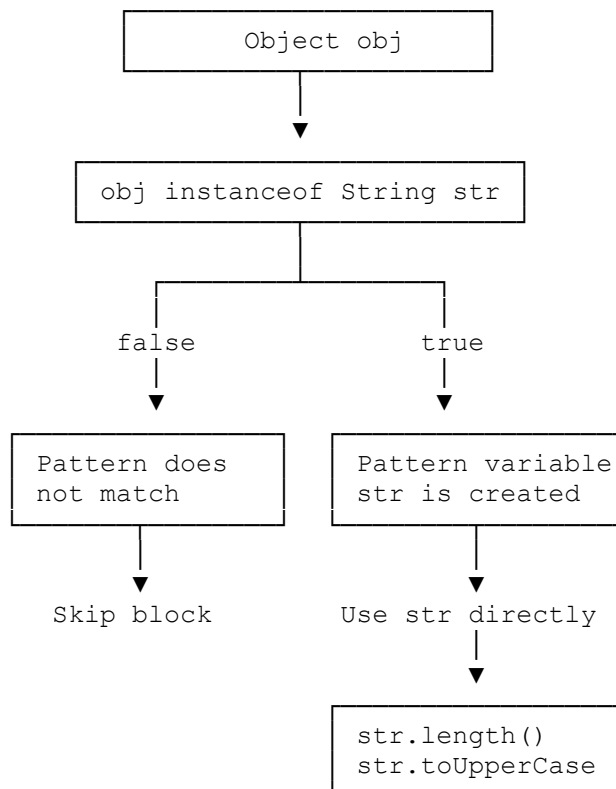
Here:

`obj instanceof String str`

means:

Check whether obj is a String, and if it is, bind the value to the pattern variable str.

Flowchart



Java Code Example

```
public class InstanceofFinal {  
  
    public static void main(String[] args) {  
  
        Object value = "Hello Java 16";  
  
        // Pattern matching for instanceof  
        if (value instanceof String str) {  
  
            System.out.println("It is a String");  
            System.out.println("Value   : " + str);  
            System.out.println("Length : " + str.length());  
        }  
    }  
}
```

Output

```
It is a String  
Value   : Hello Java 16  
Length : 13
```

Traditional vs Pattern Matching

Before Java 16

```
Object value = "Hello";  
  
if (value instanceof String) {  
    String str = (String) value;  
  
    System.out.println(str.toUpperCase());  
}
```

Java 16

```
Object value = "Hello";  
  
if (value instanceof String str) {  
    System.out.println(str.toUpperCase());  
}
```

The Java 16 version is shorter and avoids the redundant explicit cast.

Pattern Variable Scope

The pattern variable is available where the compiler knows that the pattern matched.

```
public class InstanceofFinal {  
    public static void main(String[] args) {  
        Object value = "Java";  
        if (value instanceof String str) {  
            System.out.println("Length: " + str.length());  
        }  
    }  
}
```

The variable str is available inside the if block because value has successfully matched String.

Using &&

Pattern matching works particularly well with &&:

```
Object value = "Java";  
  
if (value instanceof String str && str.length() > 3) {  
    System.out.println("String: " + str);  
}
```

The second condition can safely use str because it is evaluated only when the instanceof pattern matches.

Multiple Types Example

```
public class InstanceofFinal {  
    public static void printValue(Object value) {  
        if (value instanceof String str) {  
            System.out.println("String: " + str);  
        } else if (value instanceof Integer number) {  
            System.out.println("Integer: " + number);  
        } else {  
            System.out.println("Unknown type");  
        }  
    }  
  
    public static void main(String[] args) {
```

```

        printValue("Java");
        printValue(100);
        printValue(10.5);
    }
}

```

Output

```

String: Java
Integer: 100
Unknown type

```

Key Benefits

Feature	Traditional instanceof	Pattern Matching
Type check	Required	Required
Explicit cast	Required	Not required
Pattern variable	No	Yes
Boilerplate	More	Less
Final in	—	Java 16

In short: Java 16's instanceof pattern matching combines **type checking + casting + variable declaration** into one concise operation.